

Assignment 3: Model-Based Testing

Deadline: **May 19, 2026, 23:55**

The goal of this exercise is to extend the knowledge about testing towards approaches and tools for automatically generating test cases based on (formal) models.

Note: The assignment is based on the *RingBuffer* implementation used in Assignment-3-Coverage.

Assignment Part A: State Transition Testing

In this first part of the assignment, a state diagram is used as basis for manually designing test cases (see Black-box Testing). Then, these test cases are implemented using JUnit to create executable tests.

Step 1: State transition diagram

Draw a state transition diagram for the class *RingBuffer* example (without *RingBufferIterator*). Use the notion as presented in the lecture slides. Document your model by means of a graphical diagram.

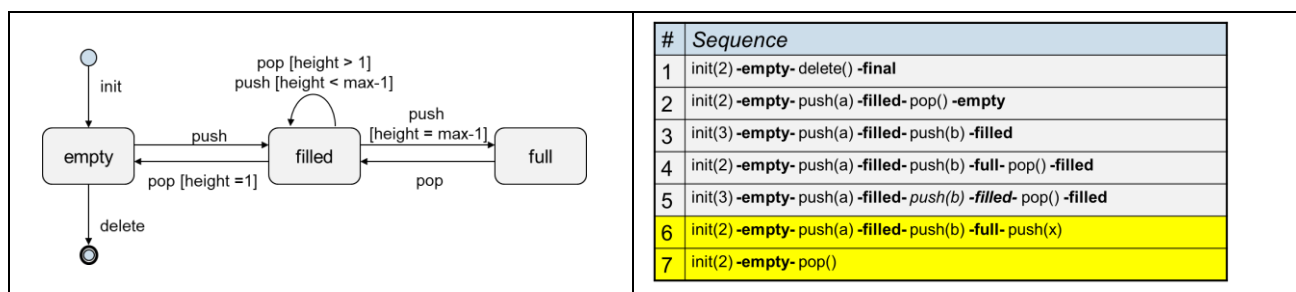
- The diagram must contain all possible states and events.
- Include invalid transitions that trigger exceptions (error cases). Note: These transitions are typically back loops, because invalid actions do not change the system's state.
- You can use an UML tool for the diagram, if you like. Yet any drawing diagram is perfectly o.k.
- Store the diagram as png-file

Step 2: Logical test cases

Based on your state transition diagram, develop logical test cases in form of sequences of possible (valid and invalid) state transitions. Store your list of test cases in the text file *RingBufferStateTransitionTests.txt*. For each test case define:

- Initial state of the test object
- Sequence of transitions containing source state, trigger event, input data, and expected target state
- Expected results or expected behavior
- Final state

Here is an example for test cases for a Stack-Implementation that allows push and pop operations:



Step 3: Implementing unit tests

Implement the derived logical test cases in form of unit tests using JUnit, where each logical test case corresponds to one unit test.

- Name the test class containing the implemented unit tests *RingBufferStateTransitionTest.java*

Assignment Part B: Model Implementation

The framework *ModelJUnit*¹ is used for generating test cases from a state model implemented as Java class. Implement an executable version of your model for *RingBuffer* named *RingBufferModel*. Use *ModelJUnit* to generate sequences exploring your model.

- Implement actions for all transitions related to *RingBuffer* methods, e.g., *enqueue*, *dequeue* and *peek*.
- The model should reflect the states of the *RingBuffer* (e.g., *empty*, *filled* or *full*). Member variables (e.g., *counter*) can be used to keep track of how many elements are in the buffer and to determine the current state of the buffer in the function *getState()*.
- Use guard functions to make sure that the actions *dequeue* is only called if the *RingBuffer* is in a state where this action is valid (e.g., *filled* or *full*)
- Add additional actions for negative scenarios ("back loops" in the diagram) like *dequeueFromEmptyBuffer* and matching guard functions to verify that the expected exceptions are thrown.

Run the model via the associated JUnit test *RingBufferModelTest*. Copy the console output into a text file named *RingBufferModel_Output.txt* for submission.

Assignment Part C: Model with Adapter

Extend the *RingBufferModel* with an adapter to actually call the methods of the class *RingBuffer*. Name the resulting model *RingBufferModelWithAdapter*.

- The model should extend the existing actions with calls to the corresponding methods of the class *RingBuffer*, and the resulting response (i.e., returned values, thrown exceptions) should be compared to the expected response.
- Make use of the model's state variable (*counter*) to assert the correct response; use JUnit asserts
- In case of expected exceptions, make sure that (1) exceptions are actually thrown, and (2) they are of the correct type and contain the correct message.
- Furthermore, provide actions for *size* and *isEmpty*, which should call the corresponding method of the class *RingBuffer*. They will not change the state of the *RingBuffer*. However, they allow asserting the response by comparing it to the counter variable whenever triggered.
- Write a unit test *RingBufferModelWithAdapterTest* to exercise *RingBufferModelWithAdapter*.

Run the test and measure the code coverage. Experiment with different lengths for the generated test sequences – set as parameter of *tester.generate()* – to achieve a maximum coverage of the called methods. Include the coverage report in the submission.

Submission & Presentation

Hand in the source code of your tests and any related files you created as zip archive and **upload it to the Moodle course platform before the deadline**.

Work in teams. Only one upload per team is required, i.e., one team member uploads the files to Moodle.

The work has to be **presented in the lecture on Thr, 11. June 2026**. Each team has to be prepared to present their results. The teams that present will be selected at the beginning of the lecture. No slides (ppt etc.) have to be prepared, just bring a laptop with a development environment including all the tools you used and the source code of your solution.

¹ For details see: M. Utting and B. Legeard: Practical Model-Based Testing: A Tools Approach. Elsevier, 2007.